Lecture 7: Processes

Professor G. Sandoval

Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.
Also some Slides by Brendan Dolan-Gavitt and Stallings: Operatings Systems. Eight Edition.

Today

- Processes
- Process States
- Unix Processes
- XV6 Processes

Virtualizing the cpu

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int
main(int argc, char *argv[])
{
        if (argc != 2) {
                fprintf(stderr, "usage: cpu <string>\n");
                exit(1);
        }
        char *str = argv[1];
        while (1) {
                Spin(1); // Repeatedly checks the time and returns once it has run for a second
                printf("%s\n", str);
        }
        return 0;
}
```

Simple Example(`cpu.c`): Code That Loops and Prints

Virtualizing the cpu (cont)

Execution result 2.

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

Virtualizing the cpu (cont)

Execution result 2.

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

Even though we have only one processor, all four of programs seem to be running at the same time!

Challenge

- Provide the illusion of multiple CPUs
- More processes than processors
- Imagine: Your laptop has two processors but you want to run 3 programs: editor, compiler, word proc.
- Need to multiplex N processors among M processes
- This is called: Timesharing, scheduling, context switching

Goals for Solution

- Transparent to user processes
- Pre-emptive for user processes
- Pre-emptive for Kernel to keep system responsive

The Abstraction: Process

- A *process* is the basic unit of execution and an abstraction for a running program.
- Splitting up the execution of a system into processes allows:
  - Isolation between programs
  - The ability to simulate running multiple programs at once with only one CPU

Processes in Parallel

- It is confusing to reason about many events happening at once
- The process model lets us consider each process as a single, *sequential* execution
- (For now, we'll consider just one CPU)

Processes != Programs

- A *program* = instructions + data
- A *process* is the OS abstraction for a running program, its resources, input/output, etc.
- Process = memory (instructions + data + stack)
- Think of a process as an instance of a program, just like an Object is an instance of a class.
- So: multiple copies of a program can be running in different processes

Process

- Each process has it's own Machine State:
  - Memory
  - Registers (PC, SP, FP, etc)
  - Persistent storage (I/O)

Lots of Processes

Activity Monitor (All Processes)

CPU | Memory | Energy | Disk | Network | Search

Process Name	% CPU	CPU Time	Threads	Idle Wake Ups	PID	User
WindowServer	6.2	31:54.97	6	8	164	_windowserver
hidd	3.6	8:27.61	8	0	98	root
Activity Monitor	3.3	0.53	14	2	4934	moyix
Scroll Reverser	2.4	3:41.98	4	0	591	moyix
kernel_task	2.1	25:34.51	110	121	0	root
launchservicesd	1.6	26.48	14	1	75	root
sysmond	0.9	3.17	6	0	203	root
https://pactwebserial.wordpress.com	0.8	28.21	20	5	4445	moyix
Dock	0.5	27.71	7	1	342	moyix
1Password mini	0.3	2:44.25	11	1	570	moyix
Keynote	0.2	21.08	13	6	4711	moyix
Preview	0.1	1:17.03	9	6	4762	moyix
galileod	0.1	2:16.57	20	2	50	root
cfprefsd	0.1	7.61	11	0	317	moyix
mosh-client	0.1	1:07.42	1	0	3145	moyix
mosh-client	0.1	1:29.61	1	0	1114	moyix
launchd	0.0	1:59.49	10	0	1	root
Dropbox	0.0	2:01.34	59	0	604	moyix
fseventsd	0.0	33.20	11	1	43	root
Seil	0.0	32.70	4	1	605	moyix
mysqld	0.0	21.00	21	3	90	_mysql
notifyd	0.0	19.65	3	0	100	root
Safari	0.0	6:26.08	20	0	890	moyix

System:	1.22%	CPU LOAD	Threads:	1288
User:	1.81%		Processes:	252
Idle:	96.96%			

Today

- Processes
- Process States
- Unix Processes
- XV6 Processes

Process API

- Create
- Destroy
- Wait
- Status
- Miscellaneous Control

Process Creation

- Processes can be created in many circumstances
  - At boot time
  - User request (e.g., typing a command)
  - A running program creates a new process on its own
- All of these are actually carried out by execution of a system call (with the exception of the *first* process, which is created directly by the kernel)

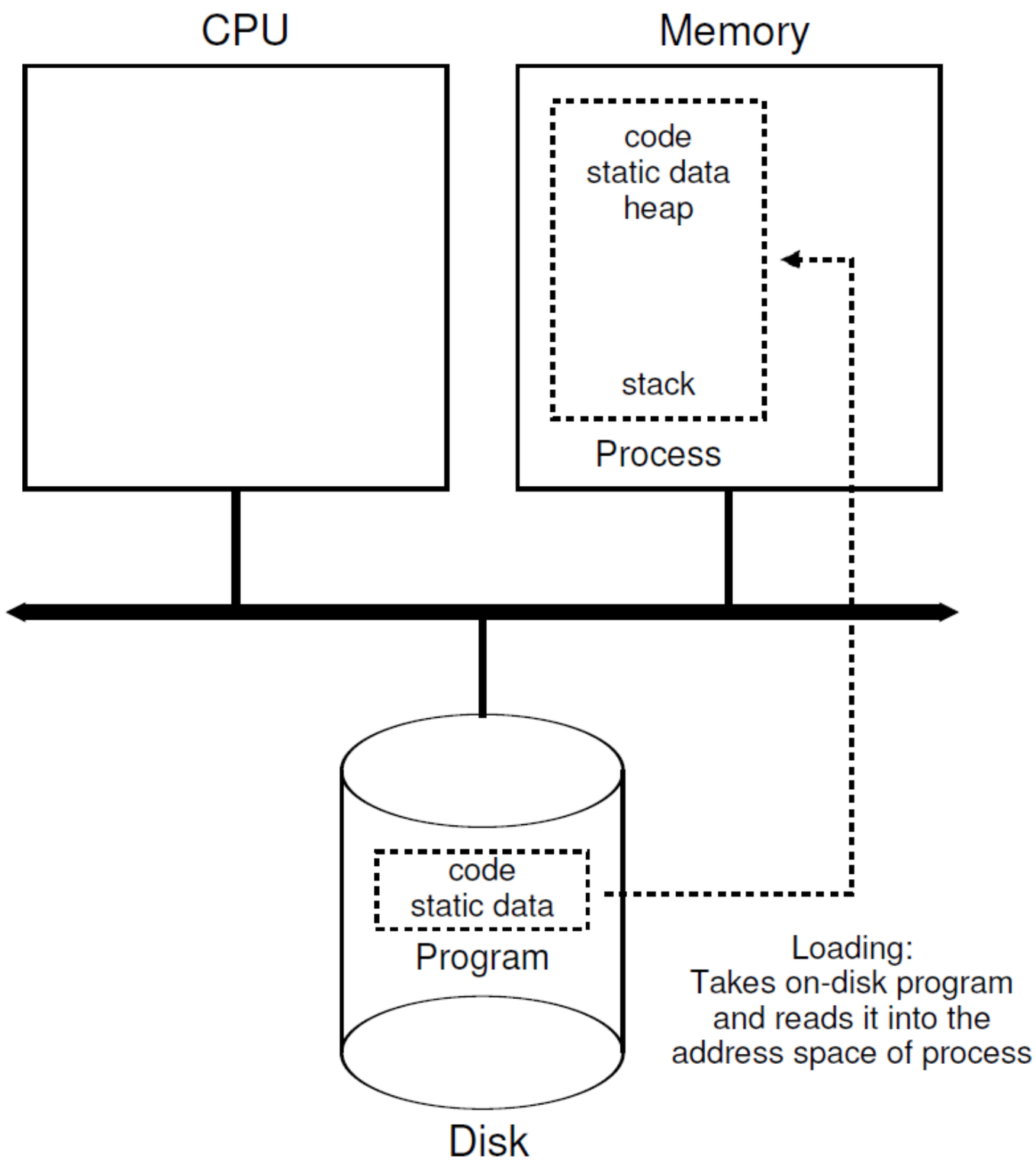


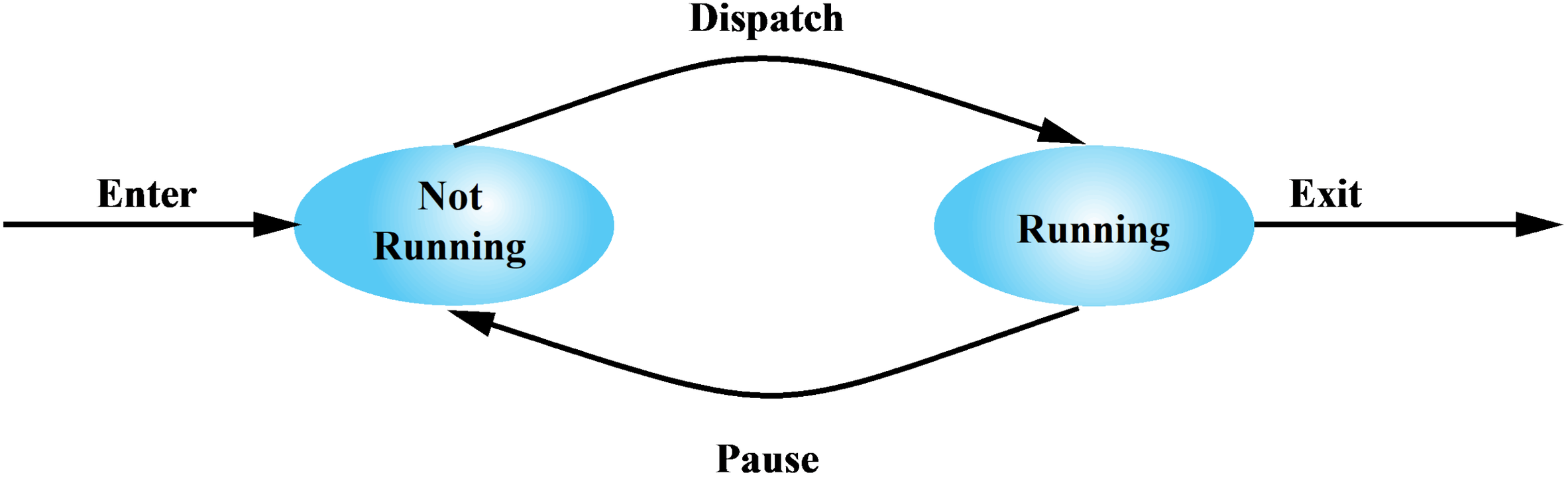
Figure 4.1: **Loading: From Program To Process**

Source: OS in 3 easy pieces.

Process Termination

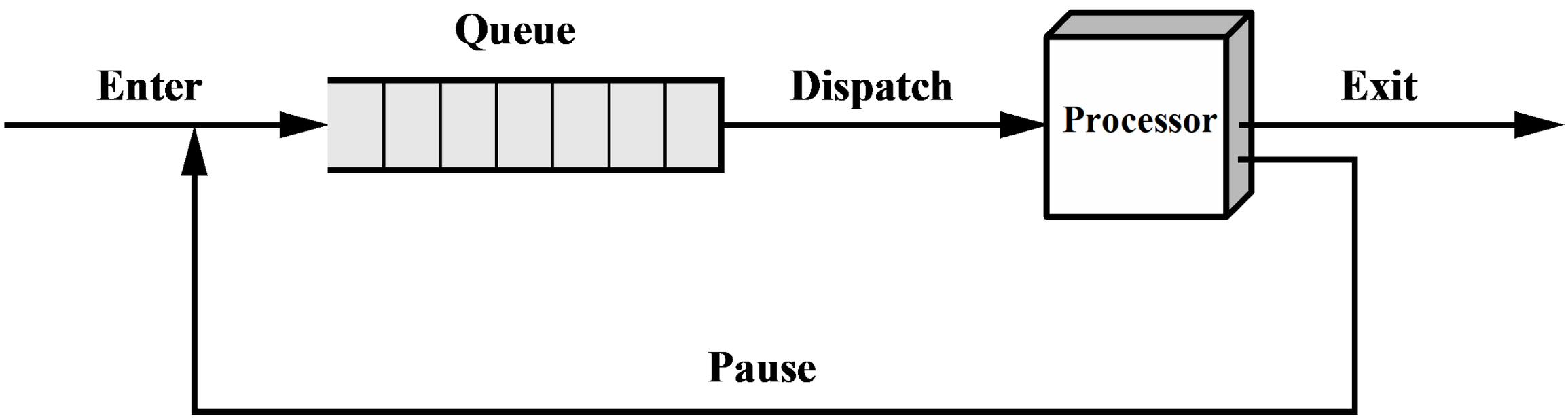
- There must be a means for a process to indicate its completion
- A batch job should include a HALT instruction or an explicit OS service call for termination
- For an interactive application, the action of the user will indicate when the process is completed (e.g. log off, quitting an application)

Two-State Process Model



(a) State transition diagram

Two-State Process Model

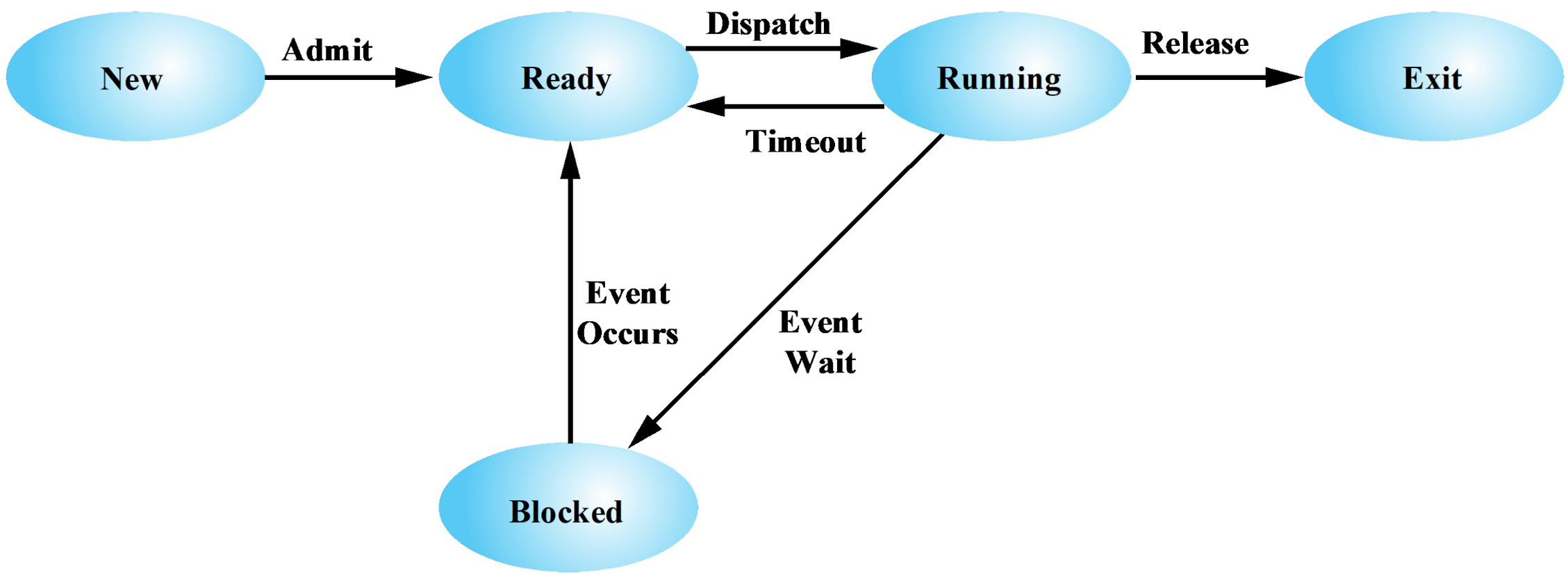


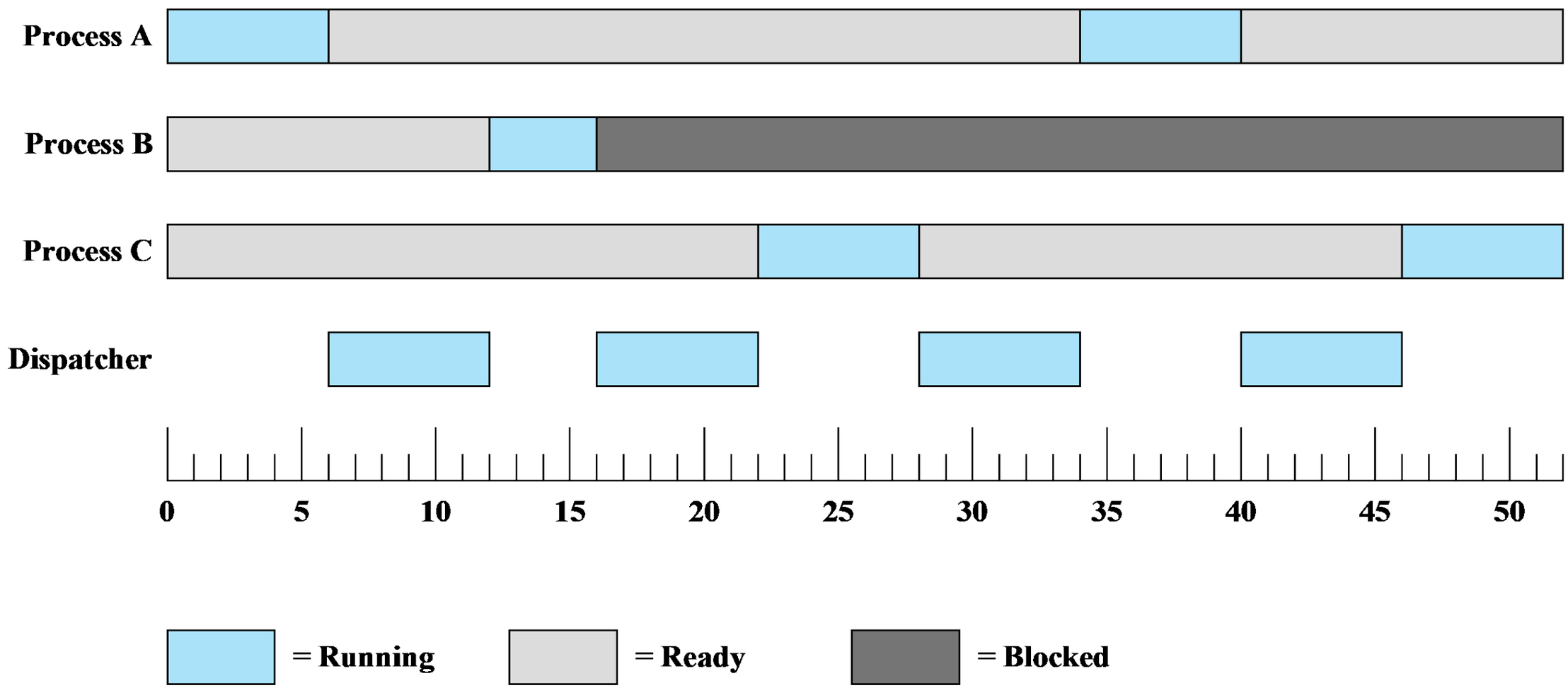
(b) Queuing diagram

Reasons for Proc Termination

- Normal Completion
- Bounds Violation
- Memory unavailable
- Others…

Five-State Process Model




Process A
Process B
Process C
Dispatcher
0
5
10
15
20
25
30
35
40
45
50
= Running
= Ready
= Blocked

Mechanisms for Interrupting the execution of a process

- System Call: Way for user program to ask for OS service
- Exception: Illegal program action. Eg. Divide by zero, access memory for PTE not present, etc..
- Interrupt: Signal generated by hardware device. Eg. Clock chip, keyboard, etc.

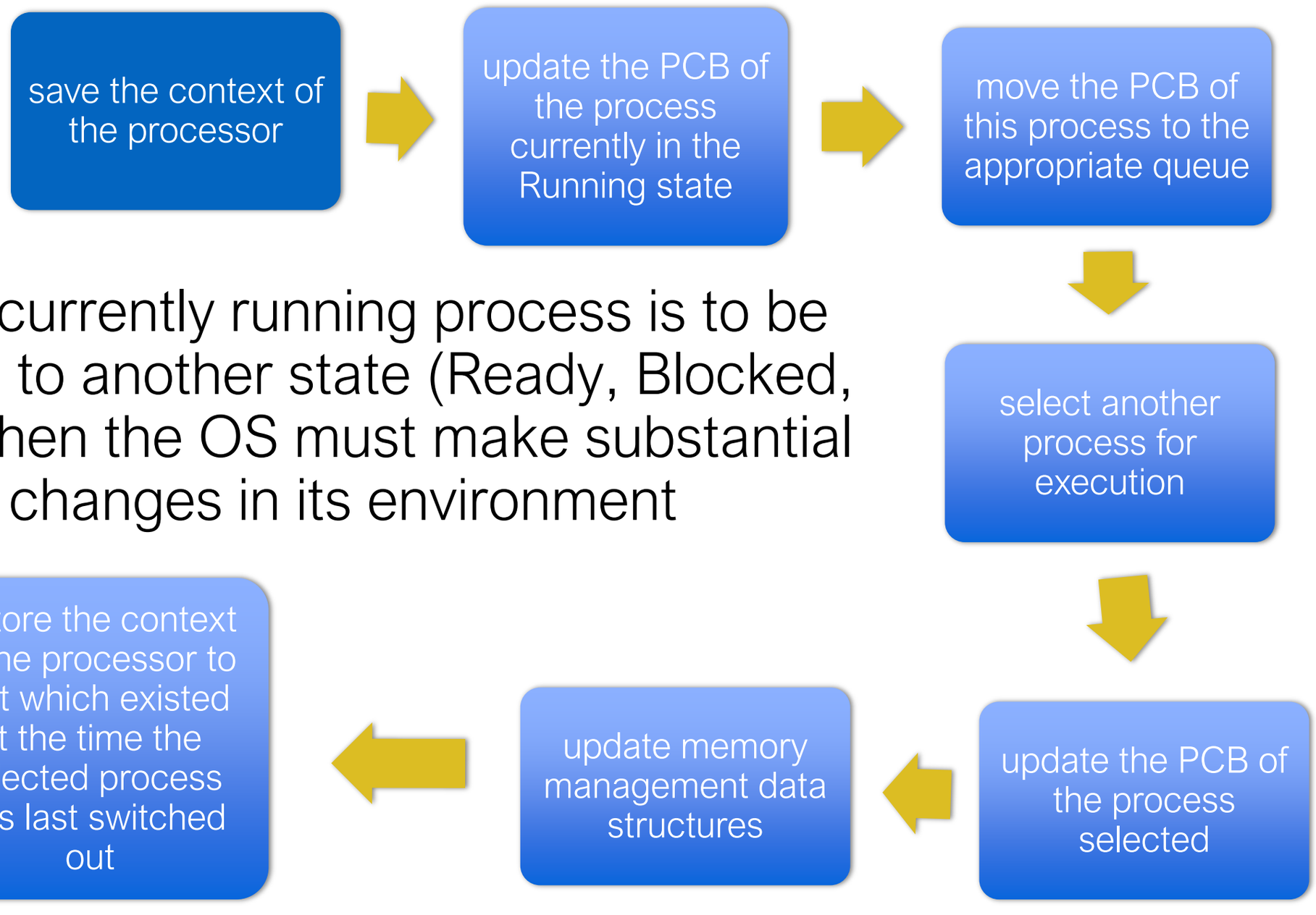
Role of the Process Control Block

The most important data structure in an OS

- contains all of the information about a process that is needed by the OS
- blocks are read and/or modified by virtually every module in the OS
- defines the state of the OS

Change of Process State

The steps in a full process switch are:



If the currently running process is to be moved to another state (Ready, Blocked, etc.), then the OS must make substantial changes in its environment

Today

- Processes
- Process States
- Unix Processes
- XV6 Processes

Process Hierarchy (UNIX)

- In UNIX, each process (aside from init) has a *parent process* that created it
- This relationship creates a tree structure, rooted at the init process
- Processes are often managed as a group consisting of a single process and all its descendants
  - E.g., signals (such as SIGTERM, to request termination) are delivered to parent and all its descendants

Process Tree

```
systemd─┬─ModemManager─┬─{gdbus}
        │              └─{gmain}
        ├─acpid
        ├─agetty
        ├─cron
        ├─dbus-daemon
        ├─dhclient
        ├─fail2ban-server───2*[{fail2ban-server}]
        ├─master─┬─pickup
        │        └─qmgr
        ├─mosh-server───bash───tmux
        ├─9*[mosh-server───bash]
        ├─ntpd
        ├─polkitd─┬─{gdbus}
        │         └─{gmain}
```

Killing Processes

- Processes can exit on their own using a system call
- One process can request that the OS terminate (*kill*) another process
  - Depending on the privilege of the requestor, this may or may not be granted

Killing Processes

- In most operating systems, process termination notifies the process to be terminated first so it can cleanly free resources
- More forceful methods exist:
  - kill -9
  - taskkill /f
- In all cases, the operating system will free the resources that it has granted to the process

Hierarchy and Termination (UNIX)

- If the parent process exits before its children, those children are said to be *orphaned*
- Orphaned child processes are *adopted* by init

Zombies (UNIX)

- *Zombies* are processes that have exited but still have an entry in the process table
- If a child process exits and the parent has not called wait() to get its exit status, the child becomes a zombie
- This is because the exit status is stored by the kernel-side data structure, and the kernel can't free it until it knows no one needs the exit status

```c
#include <stdlib.h>
#include <unistd.h>
#include <stdbool.h>

int main() {
    if (fork() == 0) {
        // Exit immediately in the child
        exit(1);
    }
    // Sleep forever without calling wait()
    // in the parent
    while(true);
    return 0;
}
```

```
USER     PID  %CPU %MEM      VSZ    RSS   TT  STAT STARTED      TIME COMMAND
moyix   7622 100.0  0.0  2432748    548 s004  R+    8:05AM   0:15.23 ./zombo
moyix   7623   0.0  0.0        0      0 s004  Z+    8:05AM   0:00.00 (zombo)
```

zombie

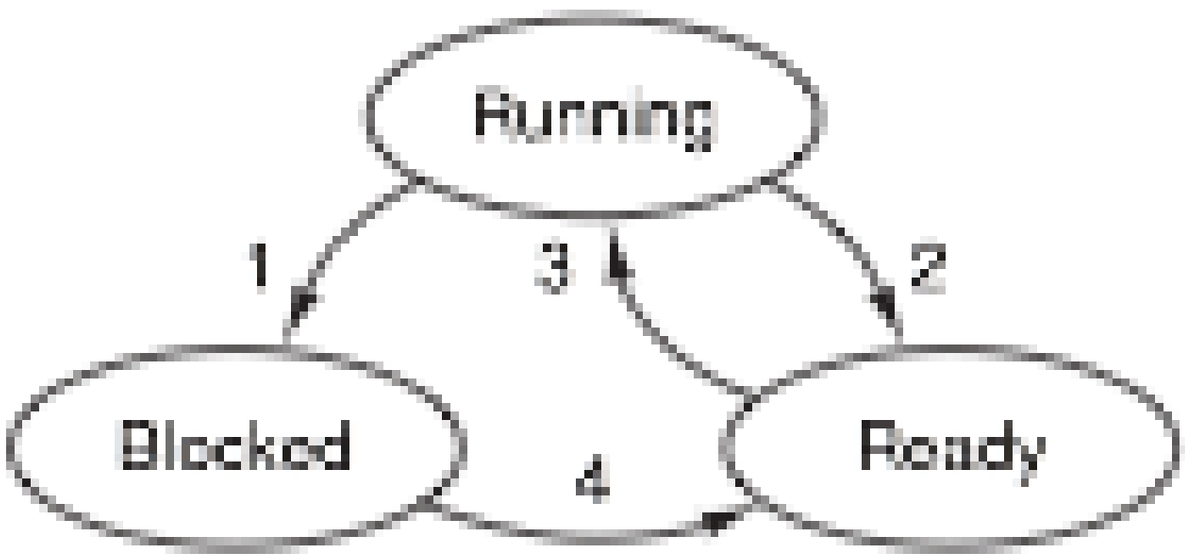
Process ~~Hierarchy~~ Anarchy (Windows)

- Windows does keep track of parent/child relationships
- These don't actually affect anything though
- Instead, when a process is created its parent gets a *handle* – a token that it can use to control the child
- Handles can be passed to other processes, and a process with sufficient privilege can obtain a handle to another process with OpenProcess

Process States

- Processes can generally be in one of three states:
  - *Running* – actively using the CPU right now
  - *Ready* – not currently running
  - *Blocked* – waiting for I/O
    - Even if the OS scheduler wants to run this process, it can't – nothing for it to do

Transitions



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

I/O Waiting

- What causes processes to block?
  - Waiting for input
  - Explicit sleep
- The kinds of input can be things like network I/O, disk, waiting for the user to click on something, etc.

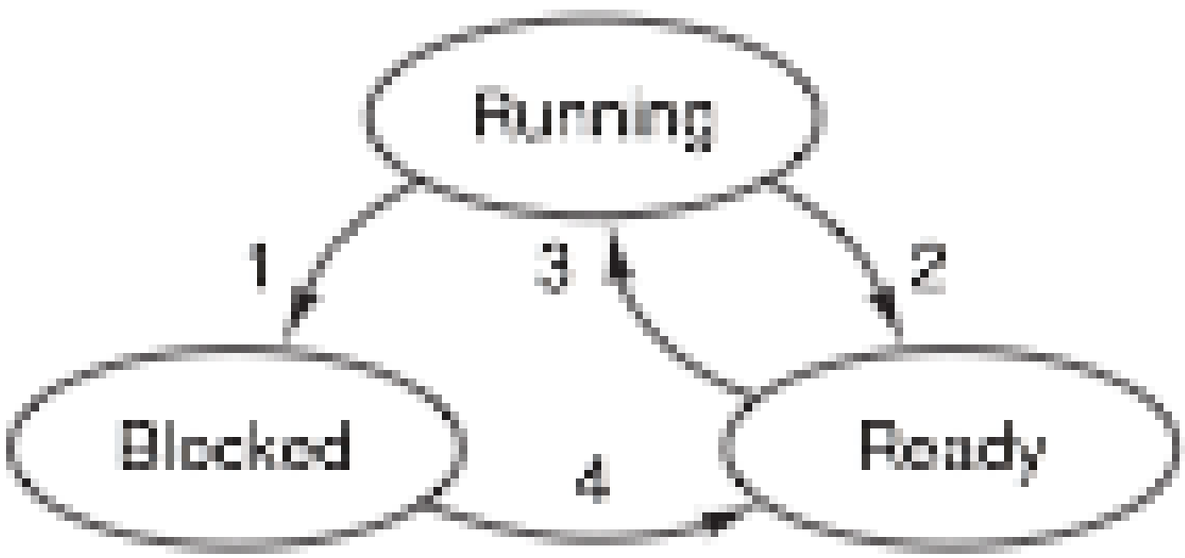
Process Anatomy: Kernel

- The kernel **tracks** what processes are active in a *process table*
  - Note: in most OSes, this is not a literal array, but rather something like a doubly-linked list
- Each process gets a structure bit of metadata called the *process control block (PCB)*

Today

- Processes
- Process States
- Unix Processes
- XV6 Processes

Transitions



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

What is the process control block?

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Process Control Block in xv6

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

xv6 Process Table

```
struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;
```

Note: NPROC = 64

Process Threads (aside)

- Each process has a thread of execution that executes the process instructions.
- Each thread has a PC and CPU State(Registers)
- Thread can be suspended and later resumed.
- Much of the state of a thread (local vars, ret address) stored on thread's stack.
- Each process has 2 stacks: user-stack and kernel stack (p->kstack)

Process Threads

- Process executing user instructions => user stack in use, kernel stack empty
- Process enters kernel thru sys_call or interrupt => kernel executes on the process's kernel stack. User stack contains saved data but not actively used.
- Kernel stack is separate and protected from user code.

Process Threads

- When a process makes a system call:
  - processor switches to kernel stack
  - raises hardware privilege level.
  - Starts executing the kernel instructions that implement the system call.

Process Threads

- When System Call Completes:
  - Kernel returns to user space
  - hardware lowers privilege level.
  - Switches back to user stack
  - Resumes executing user instructions just after the system call

Kernel Stack

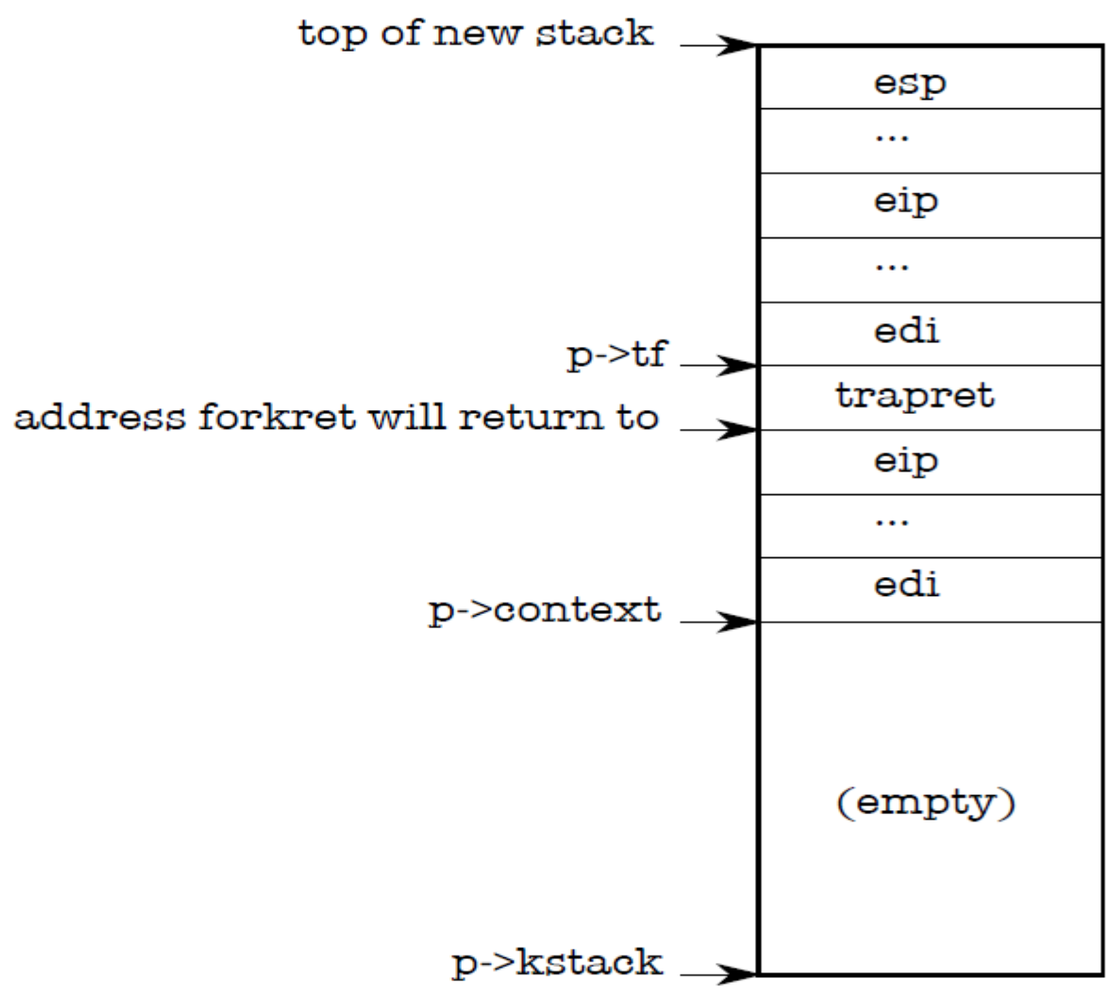
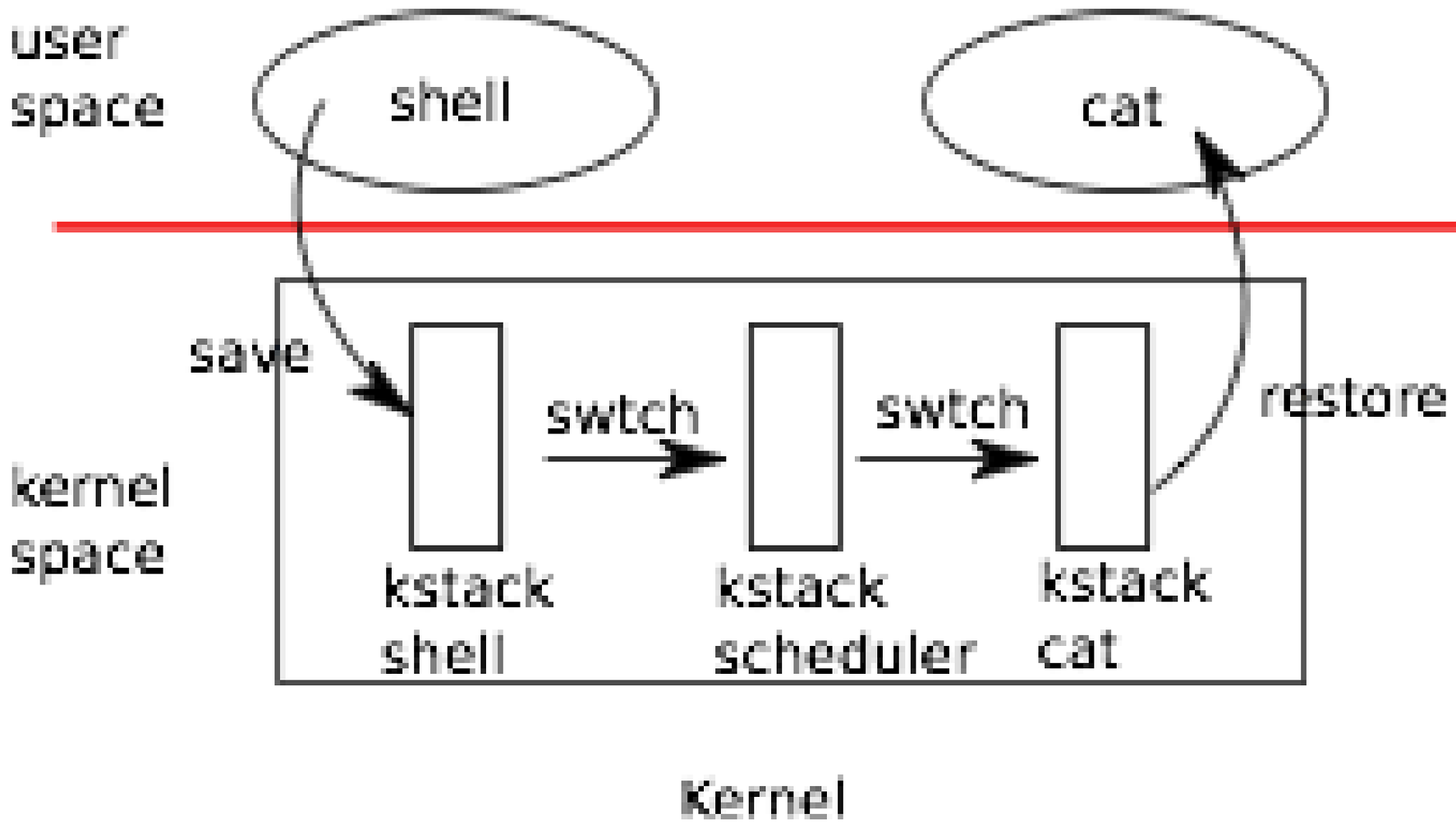
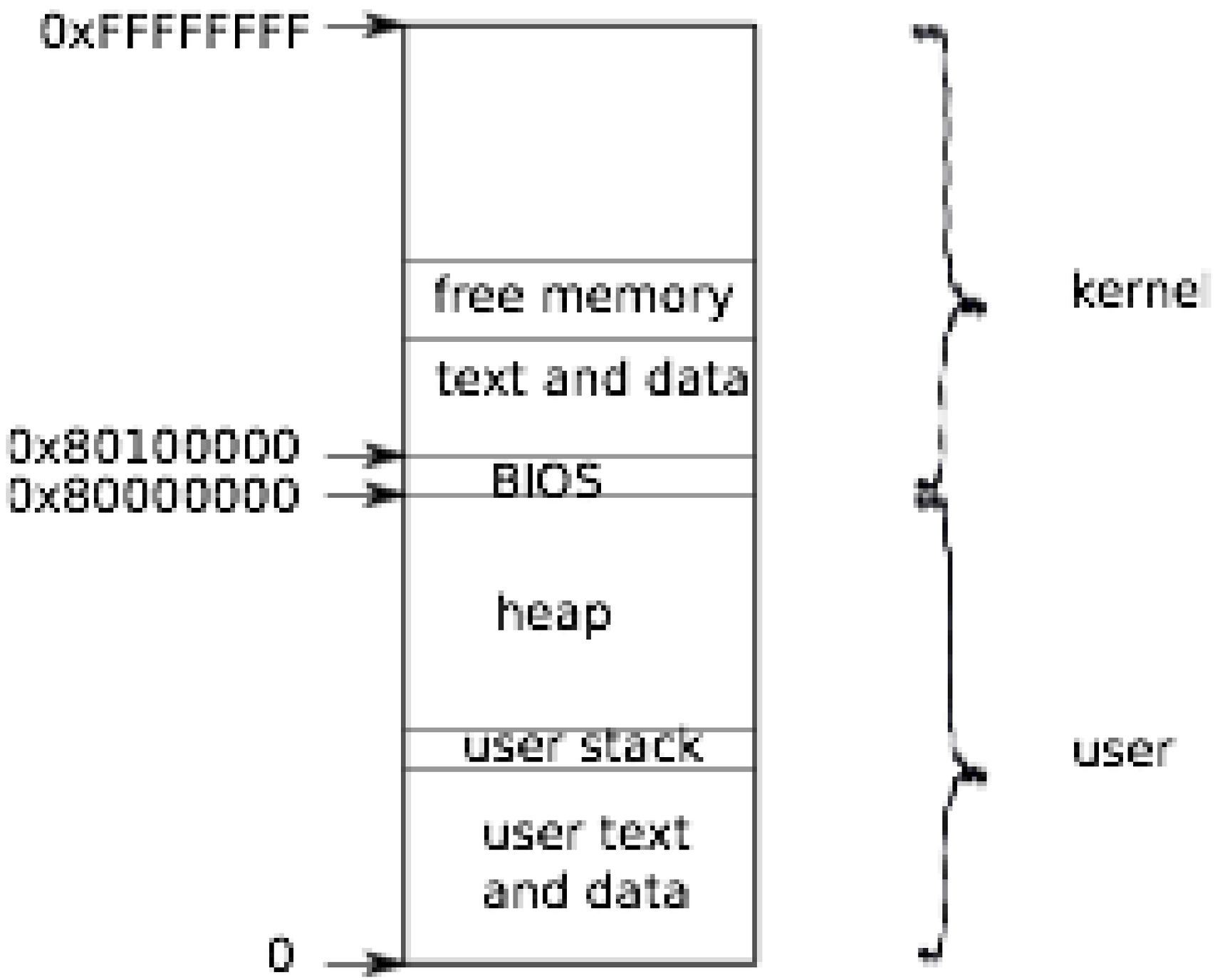


Figure 1-4. A new kernel stack.


user
space
shell
cat
save
swtch
swtch
restore
kernel
space
kstack
shell
kstack
scheduler
kstack
cat
Kernel

Create Proc

```c
// Look in the process table for an UNUSED proc.
// If found, change state to EMBRYO and initialize
// state required to run in the kernel.
// Otherwise return 0.
static struct proc*
allocproc(void)
{
  struct proc *p;
  char *sp;

  acquire(&ptable.lock);
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == UNUSED)
      goto found;
  release(&ptable.lock);
  return 0;

found:
  p->state = EMBRYO;
  p->pid = nextpid++;
  release(&ptable.lock);

  // Allocate kernel stack.
  if((p->kstack = kalloc()) == 0){
    p->state = UNUSED;
    return 0;
  }
  sp = p->kstack + KSTACKSIZE;
```

```c
  // Allocate kernel stack.
  if((p->kstack = kalloc()) == 0){
    p->state = UNUSED;
    return 0;
  }
  sp = p->kstack + KSTACKSIZE;

  // Leave room for trap frame.
  sp -= sizeof *p->tf;
  p->tf = (struct trapframe*)sp;

  // Set up new context to start executing at forkret,
  // which returns to trapret.
  sp -= 4;
  *(uint*)sp = (uint)trapret;

  sp -= sizeof *p->context;
  p->context = (struct context*)sp;
  memset(p->context, 0, sizeof *p->context);
  p->context->eip = (uint)forkret;

  return p;
}
```

xv6 Address Space Layout